Day 30 : Valid Parenthesis

✅ Problem Statement

Given a string s containing only:

( ) { } [ ]

- ✓ Check karo ki string valid hai ya nahi.
- ✓ Valid hone ki conditions:
- ✓ Opening bracket ka proper closing hona chahiye
- ✓ Order correct hona chahiye
- ✓ Har opening ka matching closing hona chahiye

📌 Example

Input: "()"
Output: true

Input: "()[]{}"
Output: true

Input: "(]"
Output: false

Input: "([)]"
Output: false

Input: "{[]}"
Output: true

🧠 Approach (Stack Use karenge)

👉 Stack ka use karke easily solve hota hai

Logic:

- ✓ Agar opening bracket mile → stack me push karo
- ✓ Agar closing bracket mile →
- ✓ stack empty hai → invalid
- ✓ top check karo match ho raha ya nahi
- ✓ End me stack empty hona chahiye

🔁 Dry Run (Important)

Example: "([{}])"

Step	Char	Stack
1	(	(
2	[	([
3	{	([{
4	}	([
5	]	(
6	)	empty

✅ Final stack empty → VALID

❌ Invalid Example: "([)]"

Step	Char	Stack
1	(	(
2	[	([
3	)	❌ mismatch

💻 Java Code (Clean + Interview Ready)

```java
package Day_30;

import java.util.Stack;

public class ValidParenthesis {

    public static boolean isValid(String s) {
        Stack<Character> stack = new Stack<>();

        for (char ch : s.toCharArray()) {

            // Opening brackets
            if (ch == '(' || ch == '{' || ch == '[') {
                stack.push(ch);
            }
            // Closing brackets
            else {
                if (stack.isEmpty()) return false;

                char top = stack.pop();

                if (ch == ')' && top != '(') return false;
                if (ch == '}' && top != '{') return false;
                if (ch == ']' && top != '[') return false;
            }
        }

        return stack.isEmpty();
    }

    public static void main(String[] args) {
        String s = "{[]}";
        String str ="{(})";
        System.out.println(isValid(s));   //true
        System.out.println(isValid(str));   //false
    }
}
```

⏱️ Complexity

Time: O(n)

Space: O(n)